Smart Blind Assistance Cane System

Full Circuit Wiring, Pin Connections & Arduino Source Code

Embedded Systems Semester Project

1. Project Overview

This document provides a complete technical reference for building the Smart Blind Assistance Cane — a wearable embedded system that attaches to a standard walking cane and provides real-time multi-sensor hazard detection for visually impaired users.

The system uses an Arduino UNO as its central controller, fusing data from four sensors (ultrasonic, flame, smoke, temperature/humidity) into a three-pattern alert system delivered through a buzzer and vibration motor. A Bluetooth module streams live data to a mobile companion app, and an RTC module timestamps every hazard event for logging.

2. Full Components List

Component	Model / Spec	Quantity
Arduino UNO	Rev3 ATmega328P	1
Ultrasonic Sensor	HC-SR04	1
Flame Sensor	Flame Sensor Module (DO output)	1
Smoke Sensor	MQ-2 or generic (AO output)	1
Temperature & Humidity	DHT11 Module	1
Bluetooth Module	HC-05 (6-pin)	1
Real Time Clock	DS1302 RTC Module	1
Buzzer	5V Passive Buzzer	1
Vibration Motor	Small DC vibration motor	1
NPN Transistor	2N2222 or BC547 (for motor)	1
Resistors	330 ohm (LEDs), 1k ohm (transistor base)	3x / 1x
LEDs	Red, Green, Yellow 5mm	1 each
Breadboard	400 tie point	1
Jumper Wires	Male-Male & Male-Female 20cm	As needed
9V Battery or Power Bank	Power supply for portable use	1
Arduino USB Cable	For programming	1

3. Pin Connections — Full Description

3.1 HC-SR04 Ultrasonic Sensor

The ultrasonic sensor measures the distance to the nearest obstacle ahead. It sends a sound pulse and measures how long the echo takes to return. Place it on the front of the cane mount, facing forward.

Component Pin	Arduino Pin	Wire Color	Description
VCC	5V	Red	Power supply from Arduino 5V pin
GND	GND	Black	Common ground on breadboard
TRIG	D2	Blue	Trigger pin — Arduino sends pulse here
ECHO	D3	Green	Echo pin — returns pulse duration to Arduino

3.2 Flame Sensor Module

The flame sensor detects infrared light emitted by fire. It has a digital output (DO) that goes LOW when a flame is detected within range (~1 meter). Mount it on the side of the cane enclosure with the sensor face outward.

Component Pin	Arduino Pin	Wire Color	Description
VCC	5V	Red	Power — connect to 5V rail on breadboard
GND	GND	Black	Ground — connect to GND rail
DO	D4	Orange	Digital output — LOW = flame detected

NOTE: Some flame sensor modules also have an AO (analog) pin — you can ignore it for this project.

3.3 Smoke / Gas Sensor

The smoke sensor (MQ-2 or similar) detects combustible gases and smoke. It outputs an analog voltage proportional to gas concentration, which Arduino reads on its analog pin A0.

Component Pin	Arduino Pin	Wire Color	Description
VCC	5V	Red	Power — connect to 5V rail
GND	GND	Black	Ground — connect to GND rail
AO	A0	Yellow	Analog output — higher value = more smoke/gas

NOTE: MQ-series sensors need ~30 seconds warm-up time after power-on before readings are accurate. The code accounts for this.

3.4 DHT11 Temperature & Humidity Sensor

The DHT11 reads ambient temperature and humidity with a single data wire. It will trigger an alert if temperature exceeds a defined threshold (e.g. above 50 degrees C, which may indicate proximity to a fire).

Component Pin	Arduino Pin	Wire Color	Description
VCC / +	5V	Red	Power supply
GND / -	GND	Black	Ground
DATA / S	D5	Purple	Single-wire data communication to Arduino

NOTE: If your DHT11 is a bare 3-pin module, it already has the pull-up resistor built in. If it's a raw 4-pin IC, add a 10k resistor between DATA and VCC.

3.5 HC-05 Bluetooth Module

The HC-05 sends sensor data wirelessly to the mobile app over Bluetooth serial. We use Arduino's SoftwareSerial library to communicate with it on digital pins D11 (RX) and D12 (TX), leaving the hardware serial pins free for USB programming/debugging.

Component Pin	Arduino Pin	Wire Color	Description
VCC	5V	Red	Power supply — HC-05 runs on 3.3V internally but module has regulator
GND	GND	Black	Ground
TXD	D12	Blue	HC-05 transmits data → Arduino receives on D12
RXD	D11	Green	HC-05 receives data ← Arduino sends on D11

NOTE: The HC-05 RXD pin is 3.3V logic. Use a voltage divider (1k + 2k resistors) between Arduino D11 and HC-05 RXD for safety. Many modules tolerate 5V directly but a divider is best practice.

3.6 RTC DS1302 Real Time Clock

The DS1302 keeps accurate time even when Arduino is powered off (it has a coin cell battery). It communicates via a simple 3-wire interface (CLK, DAT, RST). Every hazard event gets timestamped and sent to the app log.

Component Pin	Arduino Pin	Wire Color	Description
VCC	5V	Red	Power supply
GND	GND	Black	Ground
CLK	D6	Yellow	Clock signal — Arduino controls timing
DAT	D7	Orange	Data line — bidirectional data transfer
RST / CE	D8	White	Reset / Chip Enable — activates the RTC for communication

3.7 Buzzer

The 5V passive buzzer produces different tones and patterns depending on the hazard type. A passive buzzer needs a PWM or tone() signal from Arduino — do not use an active buzzer without modifying the code.

Component Pin	Arduino Pin	Wire Color	Description
+ (positive)	D9	Pink	Signal pin — Arduino drives patterns with tone()
- (negative)	GND	Black	Ground

3.8 Vibration Motor

The vibration motor cannot be driven directly by an Arduino pin (not enough current). Use an NPN transistor (2N2222 or BC547) as a switch. The Arduino pin controls the transistor base, which switches the motor on and off.

Connection	Arduino / Rail	Wire	Description
Transistor BASE	D10 via 1k resistor	Blue	Arduino controls transistor base through 1k ohm resistor
Transistor COLLECTOR	Motor negative lead	Black	Motor connects between collector and 5V
Transistor EMITTER	GND	Black	Emitter goes directly to ground
Motor positive lead	5V rail	Red	Motor powered from 5V rail, switched by transistor

NOTE: Add a small flyback diode (1N4148) across the motor leads (cathode to positive) to protect the transistor from voltage spikes when the motor switches off.

3.9 LEDs (Status Indicators)

Three LEDs show system status at a glance. Each needs a 330 ohm current-limiting resistor in series with its anode.

Component Pin	Arduino Pin	Wire Color	Description
Red LED anode	D13	Red wire	Alert active — any hazard triggered
Green LED anode	D4 (shared logic)	Green wire	System OK — no hazards detected
Yellow LED anode	D10 (shared logic)	Yellow wire	Caution — proximity warning zone
All LED cathodes	GND	Black	Ground through 330 ohm resistor each

4. Complete Pin Mapping Summary

Arduino Pin	Connected To	Function
D2	HC-SR04 TRIG	Sends ultrasonic trigger pulse
D3	HC-SR04 ECHO	Receives echo duration
D4	Flame Sensor DO	Digital flame detection input
D5	DHT11 DATA	Temperature & humidity data
D6	RTC CLK	RTC clock signal
D7	RTC DAT	RTC data line
D8	RTC RST	RTC chip enable
D9	Buzzer +	PWM buzzer control
D10	Transistor BASE (motor)	Vibration motor control via transistor
D11	HC-05 RXD	Bluetooth serial TX (SoftwareSerial)
D12	HC-05 TXD	Bluetooth serial RX (SoftwareSerial)
D13	Red LED anode	Alert LED output
A0	Smoke Sensor AO	Analog smoke/gas level reading
5V	All VCC pins	Power rail for all modules
GND	All GND pins	Common ground for all modules

5. Full Arduino Source Code

Copy this entire sketch into Arduino IDE. Install required libraries first via Sketch > Include Library > Manage Libraries: search for 'DHT sensor library' by Adafruit, 'DS1302' by Rafa Couto, and 'SoftwareSerial' (built-in).

5.1 Library Installation

- DHT sensor library — by Adafruit (install from Library Manager)
- DS1302 RTC library — by Rafa Couto (install from Library Manager)
- SoftwareSerial — already built into Arduino IDE, no install needed

5.2 Complete Sketch

```
// ============================================================
// Smart Blind Assistance Cane System
// Arduino UNO — Full Source Code
// ============================================================

#include <SoftwareSerial.h>
#include <DHT.h>
#include <DS1302.h>

// ── PIN DEFINITIONS ──────────────────────────────────────
#define TRIG_PIN     2   // HC-SR04 trigger
#define ECHO_PIN     3   // HC-SR04 echo
#define FLAME_PIN    4   // Flame sensor digital output
#define DHT_PIN      5   // DHT11 data
#define RTC_CLK      6   // DS1302 clock
#define RTC_DAT      7   // DS1302 data
#define RTC_RST      8   // DS1302 reset/CE
#define BUZZER_PIN   9   // Passive buzzer
#define MOTOR_PIN   10   // Vibration motor via transistor
#define BT_TX       11   // HC-05 RX (SoftwareSerial)
#define BT_RX       12   // HC-05 TX (SoftwareSerial)
#define LED_ALERT   13   // Red LED
#define SMOKE_PIN   A0   // Smoke sensor analog input

// ── THRESHOLDS ───────────────────────────────────────────
#define DIST_WARNING    80   // cm — caution zone
#define DIST_DANGER     30   // cm — immediate stop
#define SMOKE_THRESHOLD 400  // 0-1023 analog — adjust after testing
#define TEMP_THRESHOLD  45   // degrees Celsius

// ── OBJECTS ──────────────────────────────────────────────
DHT dht(DHT_PIN, DHT11);
DS1302 rtc(RTC_RST, RTC_DAT, RTC_CLK);
SoftwareSerial bluetooth(BT_RX, BT_TX);

// ── ALERT PATTERNS ───────────────────────────────────────
// Pattern 1: Obstacle — fast beeps, speed = distance
// Pattern 2: Flame    — long-short-long
// Pattern 3: Smoke/Temp — continuous long tone
```

```
unsigned long lastSensorRead = 0;
unsigned long lastBTSend = 0;
const int SENSOR_INTERVAL = 50;   // ms
const int BT_INTERVAL = 500;       // ms

// ── SETUP ──────────────────────────────────────────────────
void setup() {
  Serial.begin(9600);
  bluetooth.begin(9600);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(FLAME_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(MOTOR_PIN, OUTPUT);
  pinMode(LED_ALERT, OUTPUT);

  dht.begin();
  rtc.halt(false);   // Start RTC
  rtc.writeProtect(false);

  // Set RTC time once (comment out after first upload)
  // rtc.setDOW(SATURDAY);
  // rtc.setTime(12, 0, 0);
  // rtc.setDate(18, 4, 2026);

  Serial.println("Smart Cane System Ready");
  delay(2000); // Sensor warm-up
}

// ── MAIN LOOP ──────────────────────────────────────────────
void loop() {
  unsigned long now = millis();

  if (now - lastSensorRead >= SENSOR_INTERVAL) {
    lastSensorRead = now;

    // Read all sensors
    long distance = readUltrasonic();
    bool flameDetected = (digitalRead(FLAME_PIN) == LOW);
    int smokeLevel = analogRead(SMOKE_PIN);
    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

    // Priority alert logic (fire/smoke override obstacle)
    if (flameDetected) {
      alertPattern2_Flame();
      logHazard("FLAME", distance);
    }
    else if (smokeLevel > SMOKE_THRESHOLD || temperature > TEMP_THRESHOLD) {
      alertPattern3_SmokeTemp();
      logHazard("SMOKE/TEMP", distance);
    }
    else if (distance > 0 && distance < DIST_DANGER) {
      alertPattern1_Obstacle(distance, true);  // Danger zone
    }
    else if (distance > 0 && distance < DIST_WARNING) {
      alertPattern1_Obstacle(distance, false); // Warning zone
    }
```

```
    else {
      // All clear
      noTone(BUZZER_PIN);
      digitalWrite(MOTOR_PIN, LOW);
      digitalWrite(LED_ALERT, LOW);
    }

    // Send data to app via Bluetooth
    if (now - lastBTSend >= BT_INTERVAL) {
      lastBTSend = now;
      sendBluetooth(distance, flameDetected, smokeLevel, temperature, humidity);
    }
  }
}

// ── ULTRASONIC DISTANCE ──────────────────────────────────────
long readUltrasonic() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  long duration = pulseIn(ECHO_PIN, HIGH, 30000);
  long distance = duration * 0.034 / 2;
  return distance;
}

// ── ALERT PATTERN 1: OBSTACLE ────────────────────────────────
// Beep speed = proportional to distance (closer = faster)
void alertPattern1_Obstacle(long dist, bool danger) {
  digitalWrite(LED_ALERT, HIGH);
  int beepDelay = map(dist, 0, DIST_WARNING, 50, 400);
  tone(BUZZER_PIN, danger ? 1000 : 700, 80);
  digitalWrite(MOTOR_PIN, danger ? HIGH : LOW);
  delay(beepDelay);
  noTone(BUZZER_PIN);
}

// ── ALERT PATTERN 2: FLAME ───────────────────────────────────
// Long - Short - Long (SOS-inspired)
void alertPattern2_Flame() {
  digitalWrite(LED_ALERT, HIGH);
  digitalWrite(MOTOR_PIN, HIGH);
  tone(BUZZER_PIN, 1500, 400); delay(450);
  tone(BUZZER_PIN, 1500, 100); delay(150);
  tone(BUZZER_PIN, 1500, 400); delay(450);
  noTone(BUZZER_PIN);
  digitalWrite(MOTOR_PIN, LOW);
}

// ── ALERT PATTERN 3: SMOKE / TEMPERATURE ─────────────────────
// Continuous urgent tone
void alertPattern3_SmokeTemp() {
  digitalWrite(LED_ALERT, HIGH);
  digitalWrite(MOTOR_PIN, HIGH);
  tone(BUZZER_PIN, 2000, 600);
  delay(700);
  noTone(BUZZER_PIN);
  delay(100);
}
```

```
// ── LOG HAZARD WITH RTC TIMESTAMP ──────────────────────────────
void logHazard(String type, long dist) {
  Time t = rtc.getTime();
  String timestamp = String(t.hour) + ":" + pad(t.min) + ":" + pad(t.sec);
  String log = "LOG|" + type + "|" + timestamp + "|" + String(dist) + "cm";
  bluetooth.println(log);
  Serial.println(log);
}

// ── SEND SENSOR DATA TO APP ────────────────────────────────────
void sendBluetooth(long dist, bool flame, int smoke, float temp, float hum) {
  String data = "DATA|";
  data += "D:" + String(dist) + "|";
  data += "F:" + String(flame ? 1 : 0) + "|";
  data += "S:" + String(smoke) + "|";
  data += "T:" + String(temp, 1) + "|";
  data += "H:" + String(hum, 1);
  bluetooth.println(data);
}

// ── HELPER: PAD SINGLE DIGIT WITH ZERO ─────────────────────────
String pad(int n) {
  return (n < 10 ? "0" : "") + String(n);
}
```

5.3 Bluetooth Data Format

The Arduino sends two types of messages over Bluetooth serial at 9600 baud:

Type	Format	Example
Sensor data	DATA|D:<cm>|F:<0/1>|S:<0-1023>|T:<C>|H:<%>	DATA|D:45|F:0|S:280|T:27.3|H:55.0
Hazard log	LOG|<TYPE>|<HH:MM:SS>|<dist>cm	LOG|FLAME|14:32:07|20cm

The mobile app (built in MIT App Inventor) parses these pipe-delimited strings and displays the values on the dashboard. Each LOG message is appended to the hazard history list with its timestamp.

6. Testing Checklist

Test each module independently before integrating everything together:

- Upload the sketch and open Serial Monitor at 9600 baud
- Wave your hand in front of HC-SR04 — distance should print and buzzer should beep
- Hold a lighter (unlit, just the sparker) near the flame sensor — FLAME log should appear
- Cover the smoke sensor briefly — smoke level analog value should rise above threshold
- DHT11 should print temperature and humidity values in serial monitor
- Pair phone to HC-05 (PIN: 1234 or 0000) and open app — DATA messages should appear
- RTC: timestamp should be correct — set the clock in setup() once, then comment it out
- Vibration motor: should buzz on danger zone distance (under 30cm)

7. Important Notes for Demo

- Power the system from a 9V battery connected to the Arduino's DC barrel jack for portable demo
- Sensors need ~30 seconds after power-on before readings stabilize
- The smoke sensor has a preheat period — it may show high readings initially
- For the cane enclosure, use a project box (~10x6x4cm) secured with velcro straps to the handle
- The vibration motor should be taped directly to the cane handle for the user to feel it in-palm
- During your presentation, show the Serial Monitor AND the mobile app simultaneously
- Demonstrate all 3 alert patterns clearly — have a lighter and a smoke source ready

End of Technical Document